Technical Audit Report

FSD-V2 Audit Report

Security, production readiness, and engineering quality assessment

Audit date: 2026-06-03

Overall assessment: Pre-production; not production-ready until high-risk remediation and exit criteria are satisfied

Severity	Open findings	Meaning
Critical	1	Immediate security or system-compromise risk
High	11	Serious security, data integrity, tenant isolation, or production reliability risk
Medium	20	Important hardening, maintainability, operational, or defense-in-depth gap
Low	1	Minor hygiene or best-practice gap

Contents

Methodology and Evidence Base

The audit combined repository inspection with build, test, dependency, static-analysis, secret-scan, container-image, and deployment-readiness checks. Findings were assessed for production impact, exploitability, tenant/data isolation risk, and operational readiness.

Evidence types considered

- Backend and frontend package manifests, lockfiles, build scripts, tests, and lint commands.
- Authentication, authorization, tenant isolation, public endpoint, integration, and database source paths.
- Dockerfiles, Docker Compose files, environment examples, and production configuration assumptions.
- Dependency audit output, Gitleaks, Semgrep, Trivy image reports, command logs, and existing audit verification notes.

Report structure

The report includes executive summary, scope, verification evidence, severity definitions, detailed findings, security/code/testing/architecture/database/DevOps reviews, risk register, action plan, evidence appendix, production-readiness exit criteria, and ownership questions.

1. Executive Summary

Audit date: 2026-06-03

Source material: repository inspection of the `fsd-v2` Node.js/NestJS implementation, follow-up backend/frontend npm audit checks, backend test/lint verification, latest Node 20/npm 10 local verification output, Gitleaks/Semgrep follow-up scans, and Trivy image reports.

The `fsd-v2` project is a standalone booking system made up of a NestJS backend, React/Vite frontend, PostgreSQL database, and Docker Compose deployment files. The application is functional: backend build passes, frontend build passes under the latest Node 20/npm 10 local verification, backend unit tests pass, and Docker Compose was previously verified to build and start the stack.

Production readiness is still not acceptable without remediation. The most important risks are backend dependency vulnerabilities, missing database migrations, fail-open tenant isolation assumptions, several authorization gaps, public endpoints that rely on optional shared secrets, incomplete production environment requirements, sparse tests, broken linting, and missing CI/CD gates.

Area	Status	Summary
Backend build	PASS	`npm run build` passed in `backend`.
Frontend build	PASS	`npm run build` passed when run with normal filesystem access; sandboxed build can fail due to access restrictions.
Backend tests	PARTIAL PASS	2 suites / 10 tests passed, but coverage is too narrow for the feature surface.
Frontend tests	FAIL	No frontend test suite was found.
Linting	FAIL	Backend lint script exists but `eslint` is not installed.
Security	HIGH RISK	Backend dependency audit and Trivy image scans report critical/high vulnerabilities; frontend production dependency audit now reports moderate React Router vulnerabilities.
Authorization	HIGH RISK	Multiple sensitive actions lack owner/admin/permission checks.
Database	HIGH RISK	No migrations; schema management relies on TypeORM synchronize.
Deployment	PARTIAL PASS	Docker build/start was verified, but health checks, runtime hardening, and production env contract are incomplete.
Documentation	PARTIAL	README and env docs exist but are incomplete or outdated for production operation.

Overall assessment: the project can be built and run, but should be treated as pre-production until the high-risk security, database, authorization, and deployment issues are addressed.

2. Audit Scope

Audited

- `fsd-v2/backend`
- `fsd-v2/frontend`
- `fsd-v2/package` structure through backend/frontend package files
- Backend and frontend `package.json` and lockfiles
- Dockerfiles and Docker Compose files
- Environment example files
- Authentication, authorization, tenant isolation, public endpoint, and integration-related source paths
- Database configuration and entity-driven schema approach
- Existing test files
- Backend and frontend production dependency audit results
- Backend unit test and backend lint command results
- Gitleaks full-history secret scan report
- Semgrep SAST report
- Trivy container image vulnerability reports

Not audited in depth

- Legacy Go implementation outside `fsd-v2`
- Production infrastructure outside this repository
- Third-party SaaS tenant configuration
- Full source-control history beyond the Gitleaks secret scan
- Live production database contents
- Runtime penetration testing against a deployed environment
- Browser-based end-to-end behavior across all workflows
- Load, stress, or performance testing
- Accessibility testing
- License/commercial compliance review
- Formal threat model workshop with system owners

3. Build and Runtime Verification

Check	Result	Evidence
Node version	PASS	Latest local verification uses `v20.20.2`, matching the backend Docker runtime and frontend Docker build-stage major version family (`node:20-alpine`).
npm version	PASS	Latest local verification uses `10.8.2`, matching the declared `npm 10.x` project engine target.
Backend dependency install	PASS with warnings	`npm ci --legacy-peer-deps` installed 948 packages; install-time audit summary reported 43 total vulnerabilities including dev dependencies.
Frontend dependency install	PASS with warnings	`npm ci --legacy-peer-deps` installed 113 packages; install-time audit summary reported 4 moderate vulnerabilities including dev dependencies.
Backend build	PASS	`npm run build` in `backend` passed.
Frontend build	PASS with warning	`npm run build` in `frontend` passed; Vite warned that the main JS chunk is larger than 500 kB after minification.
Backend unit tests	PASS but narrow	2 suites / 10 tests passed.
Frontend tests	NOT PRESENT	No frontend test files or test script found.
Backend lint	FAIL	`eslint` command is referenced but not installed.
Backend production dependency audit	FAIL	`npm audit --omit=dev` reports 31 vulnerabilities: 20 moderate, 10 high, and 1 critical.
Frontend production dependency audit	FAIL	`npm audit --omit=dev` reports 2 moderate vulnerabilities in `react-router` / `react-router-dom`; no high or critical findings.
Backend local start	FAIL without env	`JWT_SECRET` must be set to a 32+ character value.
Docker Compose config/build/up	PASS	`docker compose config`, `build`, and `up` were successful

Important context

- `fsd-v2` is not an npm workspace root. There is no `fsd-v2/package.json`; scripts live under `backend/package.json` and `frontend/package.json`.
- Root-level npm failures are not application failures. They indicate missing root orchestration.
- Docker startup used fallback/local environment values because `fsd-v2/.env` was absent during verification.

4. Severity Guidelines

Critical: system compromise, major data loss, or severe exploitability is plausible.

High: serious security, tenant isolation, data integrity, or production reliability risk.

Medium: maintainability, operational, or defense-in-depth risk that should be fixed before production hardening is considered complete.

Low: minor issue, confusing project hygiene, or best-practice gap.

Informational: observation only.

5. Findings

AUD-001: Backend production dependencies contain critical and high vulnerabilities

Severity: Critical

Description

`npm audit --omit=dev` in the backend reports 31 production vulnerabilities: 20 moderate, 10 high, and 1 critical.

Evidence

- Latest backend verification log: `fsd-v2/backend-audit-run.txt`
- Affected packages include `@nestjs/core`, `@nestjs/platform-express`, `passport-saml`, `@xmldom/xmldom`, `xml-crypto`, `xml-encryption`, `xml2js`, `bcrypt`, `lodash`, `multer`, `nodemailer`, `tar`, `uuid`, `qs`, and `js-yaml`.

Impact

The backend depends on vulnerable packages in framework, SAML/XML, email, upload/parsing, and utility paths. Some fixes require breaking upgrades or package replacement.

Recommendation

Create a dependency remediation branch. Upgrade NestJS and related packages with regression testing, replace or upgrade the vulnerable SAML/XML stack where direct fixes are unavailable, and rerun production dependency audit before release.

AUD-002: Database schema management has no migrations

Severity: High

Description

The project ships no migration system and relies on TypeORM `synchronize` for schema creation or updates.

Evidence

- `backend/src/config/database.config.ts`
- `fsd-v2/.env.example`
- `fsd-v2/docker-compose.prod.yml`
- Existing audit notes state the app ships no migrations and documents first deploy with `DB_SYNCHRONIZE=true`.

Impact

Production schema changes are not controlled, repeatable, reviewable, or safely reversible. This creates a serious data integrity and deployment risk.

Recommendation

Introduce TypeORM migrations or an equivalent migration tool. Disable `synchronize` for every non-local environment. Add migration generation, migration review, migration execution, rollback, and backup/restore documentation.

AUD-003: Tenant RLS policy is fail-open and may be ineffective with a superuser database role

Severity: High

Description

Row-level security policy allows rows when `app.current_tenant_id` is unset, and the reference database role may be a PostgreSQL superuser.

Evidence

- `backend/src/common/rls.service.ts`
- `backend/src/common/interceptors/tenant-tx.interceptor.ts`
- Existing audit notes state RLS is documented as fail-open and may be inert when connected as superuser.

Impact

Any query executed outside the intended tenant transaction path can read or modify cross-tenant data. If the application connects as superuser, RLS protections can be bypassed.

Recommendation

Use a least-privileged database role, make RLS fail closed for tenant tables, and add integration tests proving cross-tenant access is rejected.

AUD-004: Development compose publishes unsafe default credentials and local-only login behavior

Severity: High

Description

The development Compose file defaults to weak database credentials, a default JWT secret, `ALLOW_DEV_LOGIN=true`, and demo credentials.

Evidence

- `docker-compose.yml`
- `backend/src/common/seeder.service.ts`
- `backend/src/common/demo-seeder.service.ts`

Impact

If the development compose file is reused in a shared or production-like environment, default credentials and seeded demo accounts can expose the application.

Recommendation

Keep the development compose file clearly local-only. Require explicit secrets for any shared environment, and make dev login fail closed unless explicitly enabled with a local profile.

AUD-005: Public kiosk endpoints fail open when `KIOSK_TOKEN` is unset

Severity: High

Description

Kiosk state and quick booking endpoints are public and skip token validation when `KIOSK_TOKEN` is empty.

Evidence

- `backend/src/modules/kiosk/kiosk.controller.ts`
- `backend/src/modules/kiosk/kiosk.service.ts`

Impact

An unauthenticated caller can access kiosk state or create quick bookings if the environment is missing `KIOSK_TOKEN`.

Recommendation

Fail closed when `KIOSK_TOKEN` is absent outside explicit local development. Prefer per-device tokens or tenant-scoped kiosk credentials.

AUD-006: Approval delegation lacks caller eligibility checks

Severity: High

Description

Any authenticated tenant user can call the approval delegation endpoint and redirect the first pending approval step to another user.

Evidence

- `backend/src/modules/approvals/approvals.controller.ts`
- `backend/src/modules/approvals/approvals.service.ts`

Impact

A tenant user can interfere with another user's approval workflow and potentially bypass the intended approval chain.

Recommendation

Require the caller to be the current approver, eligible delegated approver, booking owner, or admin. Add negative authorization tests for non-approvers.

AUD-007: Check-in token issuance lacks owner/admin authorization

Severity: High

Description

Any authenticated tenant user who knows a booking ID can issue a check-in token for that booking.

Evidence

- `backend/src/modules/bookings/bookings.controller.ts`
- `backend/src/modules/bookings/checkin.service.ts`

Impact

Users may mint redeemable QR/check-in tokens for bookings they do not own.

Recommendation

Require booking owner/admin authorization before token issuance. Add tests for non-owner denial.

AUD-008: Recurring series cancellation lacks caller authorization and bypasses normal side effects

Severity: High

Description

Recurring-series cancellation does not pass caller identity or role to the service and bulk-updates bookings instead of using the normal cancellation path.

Evidence

- `backend/src/modules/bookings/bookings.controller.ts`
- `backend/src/modules/bookings/recurrence.service.ts`

Impact

Unauthorized users may cancel recurring bookings. Bulk updates may skip audit logs, realtime events, notification outbox records, and calendar sync cancellation.

Recommendation

Require owner/admin authorization and route recurring-series cancellation through the normal cancellation workflow or explicitly reproduce all required side effects.

AUD-009: Production integration credential encryption is required but not fully documented

Severity: High

Description

`CredentialService` requires `INTEGRATION_SECRET_KEY` unless local/test fallback is enabled, but production environment documentation and compose requirements are incomplete.

Evidence

- `backend/src/modules/integrations/credential.service.ts`
- `docker-compose.prod.yml`
- `fsd-v2/.env.example`

Impact

The documented production compose can fail at startup or integrations can become unrecoverable if an ephemeral key is used inappropriately.

Recommendation

Document and require a stable 32-byte `INTEGRATION_SECRET_KEY` in production. Add startup validation and deployment checklist coverage.

AUD-028: Full-history Gitleaks scan found committed JWTs in Playwright auth state

Severity: High

Description

A full Git history Gitleaks scan found two JWT findings in a committed Playwright authentication state file. The file is outside `fsd-v2`, but it is part of the same repository and is still present in the working tree.

Evidence

- Gitleaks 8.30.1 command: `gitleaks detect --source . --log-opts='--all' --redact=100 --report-format json --report-path .\fsd-v2\gitleaks-report.json`
- Report: `fsd-v2/gitleaks-report.json`
- Rule: `jwt`
- File: `e2e/playwright/.auth/admin.json`
- Lines: 21 and 25
- Commit: `42a10db6fdcf72c56bbf9ccf3375e041c29f99a2`
- Commit message: `e2e and nodejs version`

Impact

Committed JWTs can allow unauthorized access if the tokens are still valid, or if the same signing secret and account/session assumptions remain in use. Even expired tokens prove that generated auth state was committed and can leak sensitive session material again.

Recommendation

Immediately invalidate the affected sessions/tokens, rotate any related JWT signing secrets if these tokens could have been issued from a shared or production-like environment, remove `e2e/playwright/.auth/admin.json` from the working tree if it contains live auth state, add the auth-state path to `.gitignore`, and purge the secret from Git history if the repository is shared externally.

AUD-030: Trivy image scans found critical and high vulnerabilities in runtime images

Severity: High

Description

Container image vulnerability scans were completed with Trivy for the API, frontend, and Postgres images. All three images contain critical and/or high severity findings.

Evidence

- Trivy version: `0.71.0`
- API report: `fsd-v2/trivy-report-fsd-v2-api.json`
- Frontend report: `fsd-v2/trivy-report-fsd-v2-frontend.json`
- Postgres report: `fsd-v2/trivy-report-postgres-16-alpine.json`
- Images scanned:
- `fsd-v2-api:latest`, image ID `45d23d04bfe7`, disk usage `787MB`, content size `117MB`
- `fsd-v2-frontend:latest`, image ID `32aeb59ff8b3`, disk usage `338MB`, content size `69.5MB`
- `postgres:16-alpine`, image ID `16bc17c64a57`, disk usage `396MB`, content size `111MB`

Summary

Image	Target Types With Findings	Total	Critical	High	Medium	Low	Unknown
`fsd-v2-api:latest`	Node.js packages	56	1	30	19	6	0
`fsd-v2-frontend:latest`	Node.js packages, `esbuild` Go binary	75	2	27	38	5	3
`postgres:16-alpine`	Alpine OS package, `gosu` Go binary	37	1	15	16	2	3

Notable critical/high items

- `fsd-v2-api:latest`: critical `passport-saml` finding `CVE-2025-54419`; high findings in `@xmldom/xmldom`, `tar`, `multer`, `nodemailer`, `lodash`, `minimatch`, `glob`, and related packages.
- `fsd-v2-frontend:latest`: critical Go `stdlib` findings in `app/node_modules/@esbuild/linux-x64/bin/esbuild`; high findings in `stdlib`, `tar`, `minimatch`, `cross-spawn`, and `glob`.
- `postgres:16-alpine`: high Alpine `libxml2` finding and critical/high Go `stdlib` findings in `/usr/local/bin/gosu`.

Impact

The runtime images can carry exploitable components even when the application source builds successfully. The frontend image is especially notable because the current Dockerfile copies the built app and `node_modules` into a Node/Vite preview runtime, so development/build-time packages such as

`esbuild` remain present in the runtime image. The Postgres image also needs base-image refresh tracking because vulnerabilities can exist in OS packages and helper binaries outside npm.

Recommendation

Rebuild images from updated base images and dependencies, then rerun Trivy. For the API and frontend, use lockfile-based installs, prune dev dependencies from runtime stages, and avoid shipping frontend build tooling in production by using the nginx production Dockerfile or another static-only runtime. Update the Postgres image tag/digest when upstream fixes are available. Add Trivy or equivalent image scanning to CI with a policy for critical/high findings.

AUD-031: Booking service add-ons can be modified without owner/admin authorization

Severity: High

Description

Any authenticated tenant user who knows a booking ID can list, attach, or detach service add-ons for that booking. The booking services controller passes only tenant ID and booking ID to the service layer; it does not pass the caller identity or role, and the service layer does not verify that the caller owns the booking or has an admin role.

Evidence

- `backend/src/modules/services/services.controller.ts`
- `backend/src/modules/services/services.service.ts`

Impact

A normal tenant user can mutate another user's booking add-ons and potentially affect catering, IT setup, billing, or invoice rollups. This is a direct authorization gap on booking-adjacent state.

Recommendation

Require booking owner/admin authorization before listing or mutating booking service add-ons. Reuse the same owner/admin helper used for booking update/cancel paths, and add negative tests proving non-owners cannot attach or detach services.

AUD-010: JWTs are accepted through URL query parameters for realtime SSE

Severity: Medium

Description

The JWT strategy accepts tokens from a `token` query parameter, and the frontend appends JWTs to EventSource URLs.

Evidence

- `backend/src/modules/auth/jwt.strategy.ts`
- `frontend/src/hooks/useRealtime.ts`

Impact

Query-string tokens can leak through logs, browser history, proxy analytics, crash traces, and referrers.

Recommendation

Use short-lived one-time realtime tokens, same-origin secure cookies with CSRF controls, or another non-query transport pattern.

AUD-011: CORS is globally enabled without origin restriction

Severity: Medium

Description

The backend is created with global unrestricted CORS.

Evidence

- `backend/src/main.ts`

Impact

The API accepts browser requests from arbitrary origins. This increases exposure if bearer tokens are stored or leaked client-side.

Recommendation

Restrict allowed origins by environment and rely on same-origin nginx proxying in production where practical.

AUD-012: Rate limiting is in-memory, spoofable, and incomplete

Severity: Medium

Description

Rate limiting is process-local, trusts the left-most `X-Forwarded-For`, and does not cover every public authentication surface.

Evidence

- `backend/src/common/guards/rate-limit.guard.ts`
- `frontend/nginx.conf`
- `backend/src/modules/webauthn/webauthn.controller.ts`

Impact

Brute-force controls become inconsistent across replicas and can be bypassed by spoofing forwarding headers. Public WebAuthn login endpoints can be attacked without the same limiter coverage.

Recommendation

Use a shared rate-limit store, trust only headers set by controlled proxies, normalize client IP handling, and apply rate limits to all public auth endpoints.

AUD-013: Several DTO/controller inputs use broad or weak validation

Severity: Medium

Description

Some controllers accept any, `unknown[]`, broad `Record<string, any>`, unvalidated object fields, or UUID-like fields as plain strings.

Evidence

- `backend/src/modules/scim/scim.controller.ts`
- `backend/src/modules/sso/sso.controller.ts`
- `backend/src/modules/floor-plans/floor-plans.controller.ts`
- `backend/src/modules/location-groups/location-groups.controller.ts`
- `backend/src/modules/departments/departments.controller.ts`
- `backend/src/modules/resources/resources.controller.ts`

Impact

Weak validation increases the chance of malformed data, unexpected payload shapes, security bypasses, and runtime errors.

Recommendation

Add explicit DTOs, use `@IsUUID()` for identifiers, validate nested objects, and add negative request tests for malformed payloads.

AUD-014: Frontend stores bearer tokens in `localStorage`

Severity: Medium

Description

The frontend stores JWTs and user session data in `localStorage`.

Evidence

- `frontend/src/api/client.ts`
- `frontend/src/hooks/useAuth.ts`
- `frontend/src/main.tsx`

Impact

Any successful XSS can steal bearer tokens and impersonate users until token expiry.

Recommendation

Move authentication to hardened same-site cookies with CSRF controls, or reduce token lifetime and add stronger CSP/XSS protections if localStorage is retained.

AUD-015: Health checks, graceful shutdown, and observability are incomplete

Severity: Medium

Description

The backend has a `/health` endpoint, but compose health checks are missing for API/frontend, graceful shutdown is not enabled, and structured logging/metrics are absent.

Evidence

- `backend/src/common/health.controller.ts`
- `backend/src/main.ts`
- `docker-compose.yml`
- `docker-compose.prod.yml`

Impact

Container orchestration cannot reliably detect unhealthy app instances, and operational troubleshooting is limited.

Recommendation

Add API/frontend health checks, `app.enableShutdownHooks()`, readiness checks for database connectivity, structured logging with request IDs, and metrics.

AUD-016: CI/CD gates are absent inside `fsd-v2`

Severity: Medium

Description

No CI/CD workflow files were found for build, test, lint, audit, Docker image, or migration checks.

Evidence

- Existing audit searches found no `.github`, `Jenkinsfile`, Azure Pipelines, GitLab CI, CircleCI, or Bitbucket pipeline files inside `fsd-v2`.

Impact

Regressions can be merged without automated verification.

Recommendation

Add CI that runs backend build, frontend build, backend tests, lint, dependency audit, Docker build, and targeted authorization tests.

AUD-017: Fine-grained permission matrix is inconsistently enforced

Severity: Medium

Description

The project defines a permission catalog, but many admin controllers rely only on broad admin roles rather than `@RequirePermission`.

Evidence

- `backend/src/modules/permissions/permission-catalog.ts`
- Admin controllers across resources, departments, holidays, reports, services, broadcasts, audit, and customization modules

Impact

Changing role permissions may not actually restrict many catalogued capabilities.

Recommendation

Map every privileged route to the intended permission and add tests proving each role can and cannot perform the expected actions.

AUD-018: Graph manual sync endpoint is not admin or permission gated

Severity: Medium

Description

The Graph manual sync endpoint is authenticated but lacks admin role or permission enforcement.

Evidence

- `backend/src/modules/graph/graph.controller.ts`

Impact

Ordinary users can trigger tenant-wide Graph subscription renewal or reconciliation work.

Recommendation

Gate the route with admin role and `integration.manage` or a dedicated Graph sync permission.

AUD-019: LDAP login interpolates raw usernames into search filters

Severity: Medium

Description

LDAP login builds the directory search filter by replacing `{username}` with raw user input.

Evidence

- `backend/src/modules/sso/sso.service.ts`

Impact

LDAP filter injection can cause unexpected account matching or authentication behavior.

Recommendation

Escape usernames using an LDAP filter escaping helper before interpolation. Add tests for special characters.

AUD-020: Teams inbound webhook accepts unauthenticated activity when bot credentials are absent

Severity: Medium

Description

The public Teams inbound endpoint accepts activity without JWT validation when `BOT_APP_ID` is unset.

Evidence

- `backend/src/modules/teams/teams.controller.ts`
- `backend/src/modules/teams/teams.service.ts`

Impact

This is acceptable for local emulator testing only. If misconfigured in a shared environment, unauthenticated inbound activity can be accepted.

Recommendation

Fail closed outside explicit local mode and require bot credentials for non-local deployments.

AUD-021: Production nginx does not apply equivalent security headers to SPA responses

Severity: Medium

Description

Helmet is configured on Nest API responses, but the production frontend nginx serves the SPA document/assets without equivalent security headers.

Evidence

- `backend/src/main.ts`
- `frontend/nginx.conf`

Impact

The first browser-loaded SPA document lacks expected CSP, frame, referrer, and related browser security headers unless set by another upstream layer.

Recommendation

Add equivalent headers at nginx or the TLS terminator, including CSP, frame restrictions, referrer policy, and HSTS when HTTPS is terminated there.

AUD-022: Swagger UI is public in all environments

Severity: Medium

Description

Swagger is registered at `/api/docs` without an environment guard or authentication gate.

Evidence

- `backend/src/main.ts`

Impact

Swagger does not bypass protected route authentication, but it exposes route inventory and DTO shapes to anyone who can reach the API host.

Recommendation

Disable Swagger in production or protect it behind admin authentication/VPN.

AUD-023: Immediate DSAR account erasure does not require recent re-authentication

Severity: Medium

Description

The self-service DSAR delete endpoint performs irreversible account anonymization/deactivation using only the current bearer token.

Evidence

- `backend/src/modules/dsar/dsar.controller.ts`
- `backend/src/modules/dsar/dsar.service.ts`

Impact

If a bearer token is stolen, an attacker can trigger destructive account erasure.

Recommendation

Require recent re-authentication, MFA confirmation, or a separate confirmation token before destructive DSAR erasure.

AUD-024: Backend lint script is non-functional

Severity: Medium

Description

backend/package.json defines `lint`, but the project does not install `eslint`.

Evidence

- backend/package.json
- Existing audit command result: `'eslint' is not recognized as an internal or external command`
- Existing audit command result: `npm ls eslint` returned an empty tree

Impact

No automated code quality gate exists despite the script.

Recommendation

Install and configure ESLint, add the lint command to CI, and fix or baseline existing lint findings.

AUD-025: Test coverage is too narrow for the application risk

Severity: Medium

Description

Only two backend unit spec files were found, covering timezone conversion and recurrence expansion. No frontend tests were found.

Evidence

- backend/src/common/tz.spec.ts
- backend/src/modules/bookings/recurrence.expand.spec.ts
- No frontend test files found under frontend/src

Impact

The current tests would not catch most security, authorization, tenant isolation, integration, or UI regressions.

Recommendation

Add focused tests for authorization, tenant isolation, booking workflows, approval workflows, public endpoints, SCIM/SSO, kiosk, webhook SSRF, and frontend critical paths.

AUD-026: Docker builds are not fully reproducible or runtime-minimal

Severity: Medium

Description

Some Dockerfiles use `npm install --legacy-peer-deps` instead of lockfile-based `npm ci`, and the backend runtime image copies full `node_modules` without pruning dev dependencies.

Evidence

- `backend/Dockerfile`
- `frontend/Dockerfile`
- `frontend/Dockerfile.prod`

Impact

Image builds can resolve different dependency versions over time, and runtime images may include unnecessary tooling and attack surface.

Recommendation

Use `npm ci` with copied lockfiles for all Docker builds. Prune dev dependencies in runtime images and run containers as non-root where practical.

AUD-029: Semgrep SAST scan found container, crypto, and nginx hardening issues

Severity: Medium

Description

A Semgrep SAST scan was run against `fsd-v2` using security-focused community rules. It scanned 281 tracked files and reported 6 blocking findings.

Evidence

- Semgrep version: `1.164.0`
- Command: `semgrep scan --config p/security-audit --config p/owasp-top-ten --config p/javascript --config p/typescript --config p/react --config p/nodejs --metrics off --disable-version-check --exclude node_modules --exclude dist --exclude build --exclude coverage --json-output .\fsd-v2\semgrep-report.json --sarif-output .\fsd-v2\semgrep-report.sarif .\fsd-v2`
- JSON report: `fsd-v2/semgrep-report.json`
- SARIF report: `fsd-v2/semgrep-report.sarif`
- `backend/Dockerfile:21`: missing non-root USER
- `frontend/Dockerfile:18`: missing non-root USER
- `backend/src/modules/integrations/credential.service.ts:100`: AES-GCM decrypt path does not pass an explicit authentication tag length to `createDecipheriv`
- `frontend/nginx.conf:28`, `frontend/nginx.conf:43`, `frontend/nginx.conf:52`: proxied `Host` header is derived from `$host`

Impact

Containers running as root increase blast radius after a container compromise. AES-GCM should enforce the expected authentication tag length to avoid accepting shorter tags. Forwarding an attacker-influenced Host header can create host-header injection risk in downstream services if any code uses host-derived URLs, redirects, tenant resolution, or security decisions.

Recommendation

Run backend and frontend containers as a non-root user. Update AES-GCM decryption to specify and enforce the expected 16-byte auth tag length. Replace forwarded nginx `Host` values with an explicit internal host value or strict allow-list behavior where the backend needs the original host.

AUD-032: Forced password-change tokens are not one-time and do not re-check account state

Severity: Medium

Description

The forced password-change flow issues a short-lived changeToken with `purpose: pwd_change`, then accepts that token until expiry. The change-password path verifies purpose and expiry, but it does not make the token one-time, does not bind it to a token version, and does not re-check that the user is still in `mustChangePassword` state before changing the password.

Evidence

- `backend/src/modules/auth/auth.service.ts`
- `backend/src/modules/users/users.service.ts`

Impact

If a change token is exposed through logs, browser state, support tooling, or local storage, it can be replayed until expiry to reset the account password again. The short expiry limits impact, but the flow is weaker than a one-time password-reset exchange.

Recommendation

Store a one-time password-change nonce or token version, consume it on successful password change, and reject tokens when `mustChangePassword` is already cleared or the user's password/token version has changed. Add tests for replay rejection and stale-token rejection.

AUD-033: Frontend production dependencies contain moderate React Router vulnerabilities

Severity: Medium

Description

`npm audit --omit=dev` in the frontend reports 2 moderate production vulnerabilities affecting `react-router-dom` through `react-router`.

Evidence

- Frontend command output provided on 2026-06-04
- `react-router` advisory: same-origin redirect with a path starting `//` can cause an open redirect through protocol-relative URL reinterpretation
- Affected range: `react-router >=6.7.0 <6.30.4`
- Affected direct dependency path: `react-router-dom`

Impact

If application routing or redirect handling accepts attacker-controlled paths, a crafted protocol-relative path can contribute to open-redirect behavior. No high or critical frontend production dependency vulnerabilities were reported in this run.

Recommendation

Upgrade `react-router-dom` to a fixed version that pulls in `react-router >=6.30.4`, rerun `npm ci --legacy-peer-deps`, `npm run build`, and `npm audit --omit=dev`, and check affected navigation/login redirect flows for regressions.

AUD-027: Project hygiene and documentation gaps can confuse handover

Severity: Low

Description

The project contains a zero-byte `docker` file, no root npm orchestration, and README claims that are outdated relative to implemented modules.

Evidence

- `fsd-v2/docker`
- No `fsd-v2/package.json`
- `fsd-v2/README.md`
- Implemented backend modules under `backend/src/modules`

Impact

New maintainers may run incorrect commands or misunderstand feature completeness.

Recommendation

Remove or explain the zero-byte file, update README, and either add root scripts or clearly document backend/frontend command locations.

6. Security Findings Summary

Critical

- Backend production dependency vulnerabilities include one critical issue and multiple high issues.

High

- Database migration absence creates production data integrity risk.
- RLS is fail-open and may be ineffective with superuser connections.
- Development defaults are dangerous outside local environments.
- Full-history Gitleaks scan found committed JWTs in Playwright auth state.
- Trivy image scans found critical/high vulnerabilities in API, frontend, and Postgres runtime images.
- Kiosk public endpoints fail open when `KIOSK_TOKEN` is unset.
- Approval delegation, check-in token issuance, recurring-series cancellation, and booking service add-on mutation have authorization gaps.
- Production integration encryption key requirements are incomplete.

Medium

- JWT query-string transport is used for realtime SSE.
- Forced password-change tokens are short-lived but replayable until expiry.
- Global CORS is unrestricted.
- Rate limiting is process-local, spoofable, and incomplete.
- Semgrep SAST found container non-root, AES-GCM tag-length, and nginx Host-header hardening issues.
- DTO validation is weak in several places.
- Frontend bearer tokens are stored in `localStorage`.
- Frontend production dependency audit reports moderate React Router open-redirect findings.
- Graph sync, LDAP login, Teams inbound webhook, Swagger exposure, DSAR erasure, nginx security headers, linting, and CI/CD need remediation.

Positive security observations

- Global JWT protection is present, with explicit `@Public()` opt-outs.
- Helmet is configured for backend API responses.
- DTO validation is enabled globally.
- Webhook URL validation includes HTTPS enforcement and SSRF-oriented host/IP checks.
- Backend startup rejects missing or short `JWT_SECRET`.

- Frontend production dependency audit currently reports no high or critical vulnerabilities.

7. Code Quality Findings

- Backend linting is currently broken because `eslint` is missing.
- Several controllers accept broad payloads that should be converted into strict DTOs.
- Fine-grained permissions are defined but not consistently enforced across controllers.
- Root project orchestration is absent, so new maintainers must know to run scripts from `backend` and `frontend`.
- README is outdated relative to the current module set.
- The application has a broad feature surface with limited automated regression coverage.

8. Testing Findings

Current state

- Backend unit tests pass: 2 suites / 10 tests.
- Test files found: `backend/src/common/tz.spec.ts` and `backend/src/modules/bookings/recurrence.expand.spec.ts`.
- No frontend test files were found.
- No coverage report was available in the prior audit.

Important missing test areas

- Authentication and MFA flows
- Role and permission enforcement
- Tenant isolation and RLS behavior
- Approval delegation and approval decisions
- Check-in token issuance and redemption
- Recurring booking cancellation side effects
- Booking service add-on authorization
- Forced password-change token replay and stale-token handling
- Kiosk public endpoint token requirements
- SCIM bearer token behavior
- SSO/SAML/OAuth/LDAP edge cases
- WebAuthn public login rate limits
- Webhook SSRF and DNS rebinding behavior

- Docker boot and production environment validation
- Frontend login, booking, approval, admin, and kiosk workflows

9. Architecture Review

Strengths

- Backend feature modules are separated by domain.
- Global guards, tenant transaction handling, and DTO validation exist.
- Realtime and notification paths are centralized enough to be identifiable.
- Docker Compose supports local and production-oriented deployment files.

Risks

- Tenant isolation depends heavily on correct transaction/context use and fail-open RLS.
- Realtime delivery appears process-local, so multi-replica deployments will not share events without Redis/pub-sub or another broker.
- Authorization logic is inconsistent across sensitive endpoints.
- Integration credential encryption is present but production configuration is incomplete.
- Audit logging is best-effort and not tamper-evident.

Recommendations

- Add explicit authorization service/helper patterns for owner/admin checks.
- Make tenant isolation fail closed and test it at database level.
- Add a shared event bus if multiple API replicas are expected.
- Strengthen audit durability and tamper-evidence for sensitive actions.

10. Database Review

Findings

- No migration system is present.
- TypeORM `synchronize` is used for schema management.
- Production compose comments document a first boot with `DB_SYNCHRONIZE=true`.
- RLS policies are fail-open and can be weakened by database role choice.
- No backup/restore strategy was found in the audited materials.

Recommendations

- Implement migrations before production release.
- Disable synchronize for all non-local environments.
- Use least-privileged DB users.
- Add schema migration CI checks.
- Add backup, restore, and rollback runbooks.
- Add index/performance review for booking search, calendar, reporting, and tenant-scoped queries.

11. DevOps and Deployment Review

Findings

- Docker Compose build/start was verified in the previous audit.
- Production compose keeps API/Postgres internal and exposes the frontend.
- API/frontend compose health checks are missing.
- Backend graceful shutdown hooks are absent.
- Docker builds are not consistently lockfile-reproducible.
- Runtime image minimization is incomplete.
- CI/CD files were not found inside `fsd-v2`.
- Production env docs are incomplete for integration encryption and some public integration secrets.

Recommendations

- Add CI/CD gates for build, test, lint, audit, Docker, and migration checks.
- Add API/frontend health checks and readiness probes.
- Add graceful shutdown.
- Use `npm ci` in Dockerfiles and prune runtime dependencies.
- Add recurring image scanning and SBOM generation.
- Add environment validation and a production deployment checklist.

12. Risk Register

ID	Severity	Status	Area
AUD-001	Critical	Open	Dependencies
AUD-002	High	Open	Database
AUD-003	High	Open	Tenant isolation
AUD-004	High	Open	Secrets/config
AUD-005	High	Open	Public endpoints
AUD-006	High	Open	Authorization
AUD-007	High	Open	Authorization
AUD-008	High	Open	Authorization
AUD-009	High	Open	Production config
AUD-028	High	Open	Secrets/history
AUD-030	High	Open	Container images
AUD-031	High	Open	Authorization
AUD-010	Medium	Open	Authentication/session
AUD-011	Medium	Open	CORS
AUD-012	Medium	Open	Rate limiting
AUD-013	Medium	Open	Validation
AUD-014	Medium	Open	Frontend session storage
AUD-015	Medium	Open	Observability/deployment
AUD-016	Medium	Open	CI/CD
AUD-017	Medium	Open	Permissions
AUD-018	Medium	Open	Integrations
AUD-019	Medium	Open	LDAP
AUD-020	Medium	Open	Teams integration
AUD-021	Medium	Open	Security headers
AUD-022	Medium	Open	Swagger exposure
AUD-023	Medium	Open	DSAR/account deletion
AUD-024	Medium	Open	Code quality
AUD-025	Medium	Open	Testing
AUD-026	Medium	Open	Docker

ID	Severity	Status	Area
AUD-029	Medium	Open	SAST/hardening
AUD-032	Medium	Open	Authentication/session
AUD-033	Medium	Open	Frontend dependencies
AUD-027	Low	Open	Documentation/hygiene

13. Recommended Action Plan

Immediate actions before production

1. Fix or replace vulnerable backend dependency paths, especially SAML/XML, NestJS, multer, nodemailer, lodash, tar, and uuid-related paths.
2. Fix authorization gaps for approval delegation, check-in token issuance, recurring-series cancellation, booking service add-on mutation, and Graph manual sync.
3. Invalidate the committed Playwright JWTs, rotate related secrets if needed, ignore generated auth-state files, and remove the leaked material from shared Git history.
4. Remediate Trivy image findings by updating dependencies/base images, pruning runtime images, replacing the frontend Node preview runtime with a static production runtime, and refreshing the Postgres image when fixes are available.
5. Introduce database migrations and disable TypeORM synchronize outside local development.
6. Make tenant isolation fail closed and run with a least-privileged DB user.
7. Make public endpoints fail closed when required secrets are missing.
8. Complete production environment requirements, including `INTEGRATION_SECRET_KEY`, kiosk tokens, Teams bot credentials, WebAuthn origin/RP settings, VAPID keys, Graph notification URL, and SMTP/integration secrets.
9. Add CI gates for backend build, frontend build, backend tests, lint, dependency audit, Docker build, and image scanning.

Short-term actions

1. Install and configure ESLint.
2. Add targeted backend tests for auth, permissions, tenant isolation, approvals, recurring bookings, booking service add-ons, check-in, kiosk, SCIM, SSO, and webhooks.
3. Add frontend tests for login, booking, approval, admin, and kiosk flows.
4. Add API/frontend health checks, graceful shutdown, structured logging, request IDs, and readiness checks.
5. Add production nginx security headers.
6. Restrict CORS by environment.
7. Move realtime authentication away from query-string JWTs.
8. Harden forced password-change tokens so they are one-time and rejected after account state changes.
9. Upgrade `react-router-dom` to a version that resolves the moderate React Router audit findings.
10. Harden rate limiting with a shared store and trusted proxy handling.

Long-term actions

1. Add tamper-evident audit logging and stronger security event retention.
2. Add a shared realtime/event bus for multi-replica deployments.
3. Add monitoring, metrics, alerting, and operational runbooks.
4. Add disaster recovery, backup/restore, and migration rollback procedures.
5. Refactor broad DTO/controller inputs into strict request contracts.
6. Maintain a recurring dependency upgrade and vulnerability review process.

14. Evidence Appendix

Area	Command / source	Result	Key output
Backend runtime	`node --version` in `backend`	PASS	`v20.20.2`
Backend npm	`npm --version` in `backend`	PASS	`10.8.2`
Backend install	`npm ci --legacy-peer-deps` in `backend`	PASS with warnings	Installed 948 packages; install-time audit summary reported 43 vulnerabilities including dev dependencies.
Backend build	`npm run build` in `backend`	PASS	`nest build` completed with no build error in `fsd-v2/backend-audit-run.txt`.
Backend tests	`npm test -- --runInBand` in `backend`	PASS	2 suites / 10 tests passed.
Backend lint	`npm run lint` in `backend`	FAIL	`eslint` is not recognized as an internal or external command.
Backend production audit	`npm audit --omit=dev --json` in `backend`	FAIL	31 vulnerabilities: 20 moderate, 10 high, 1 critical.
Frontend runtime	`node --version` in `frontend`	PASS	`v20.20.2`
Frontend npm	`npm --version` in `frontend`	PASS	`10.8.2`
Frontend install	`npm ci --legacy-peer-deps` in `frontend`	PASS with warnings	Installed 113 packages; install-time audit summary reported 4 moderate vulnerabilities including dev dependencies.
Frontend build	`npm run build` in `frontend`	PASS with warning	Vite built successfully; main JS chunk was about 998.89 kB before gzip.
Frontend production audit	`npm audit --omit=dev --json` in `frontend`	FAIL	2 moderate vulnerabilities in `react-router` / `react-router-dom`; no high or critical findings.

Notes

- The backend log contains PowerShell `NativeCommandError` formatting around npm warnings and Jest output because stderr was captured into the log. The install and test commands still completed successfully.
- The backend install-time vulnerability summary includes dev dependencies, while `npm audit --omit=dev` is the production dependency audit used for release risk.
- Frontend production runtime may be nginx when `frontend/Dockerfile.prod` is used; Node/npm versions apply to frontend build-time verification.

15. Production Readiness Exit Criteria

Do not treat the project as production-ready until all of the following are true:

1. Backend production dependency audit has no critical/high findings, or each remaining finding has a documented, time-bound risk acceptance.
2. Frontend production dependency audit has no unresolved high/critical findings, and moderate findings such as React Router are either remediated or explicitly risk-accepted.
3. Trivy image scans for API, frontend, and database images have no critical/high findings, or each remaining finding has a documented, time-bound risk acceptance.
4. Database migrations are implemented, reviewed, and exercised in CI; `DB_SYNCHRONIZE` is disabled outside local development.
5. Tenant isolation is fail-closed at database and service layers, uses a least-privileged database role, and has negative cross-tenant integration tests.
6. All High authorization gaps are fixed with owner/admin/permission checks and negative tests.
7. Public endpoints fail closed when required shared secrets or integration credentials are missing outside explicit local development.
8. Production environment validation covers JWT, integration encryption, kiosk, Teams, WebAuthn, VAPID, Graph, SMTP, and webhook settings.
9. CI runs backend build, frontend build, backend tests, lint, dependency audit, Docker build, migration checks, and image scanning.
10. API/frontend health checks, graceful shutdown, readiness checks, structured logging, request IDs, and operational metrics are in place.
11. Deployment documentation includes backup/restore, migration rollback, secret rotation, incident response, and dependency/image update procedures.

16. Ownership Summary

If taking ownership tomorrow, prioritize these questions

1. Can we safely deploy schema changes? Currently no, because migrations are absent.
2. Can one tenant access another tenant's data? Current RLS design and role assumptions need fail-closed verification.
3. Can normal users perform privileged actions? Yes, several authorization gaps are documented.
4. Can unauthenticated callers reach sensitive public endpoints? Yes, if required public endpoint secrets are missing.
5. Can we prove regressions are caught automatically? Not yet; tests and CI are too limited.
6. Can we safely operate this in production? Not yet; health checks, observability, config validation, and dependency remediation are incomplete.

The system is a strong functional prototype or internal pre-production implementation. It needs security remediation, migration discipline, authorization tests, CI/CD, and operational hardening before it should be treated as production-ready.